

GitHub Webhook을 업무 성공까지 연결

Issue #19 구현·배포·조직 Hook 전환·E2E 검증 완료 보고서

최종 판정	PR Merge와 Push 모두 GitHub 202 → Activepieces SUCCEEDED → Synology Chat success=true로 검증했습니다.
-------	---

항목	확인 결과
대상	ablecloud-team 조직의 chat.ablecloud.io Webhook
범위	Push, PR closed + merged=true
운영 상태	6/6 서비스 healthy, Worker polling ready
보안	HMAC 검증, 직접 Activepieces Webhook 차단, 저장소 Secret 0건
관련 자산	설계·계약·Flow Manifest·Runbook·검증 보고서·PDF·PPTX

문서 상태 완료

검증 기준일 2026-07-31
관련 이슈 #19 · 구현 PR #35

1. 완료 결론

기존 Synology Chat 직접 조직 Webhook을 TechFlow 수신점으로 전환했습니다. Event Gateway가 GitHub 서명 검증·이벤트 정규화·중복 억제를 담당하고, Activepieces가 시각적 Flow 실행과 Run 이력을 소유하며, 내부 Chat Adapter가 실제 메시지 게시 성공까지 판정합니다.

업무 성공 기준

HTTP 200만으로 성공 처리하지 않습니다. GitHub 접수, Flow 성공, Chat 응답의 success=true가 모두 충족되어야 완료입니다.

최종 조직 Hook 카나리

계층	PR Merge	Push
GitHub Delivery	202	202
Activepieces	SUCCEEDED	SUCCEEDED
Synology Chat	success=true	success=true

정상 동작 중인 KakaoWork 조직 Hook은 URL Host, push 이벤트, SSL 설정을 변경하지 않았습니다. 담당자 및 별도 실패 알림 채널은 아직 미정이므로 Activepieces Flow Runs와 TechFlow Observer에서 실패를 확인하도록 유지했습니다.

2. 구현 아키텍처와 책임 경계

계층	책임	주요 구현
GitHub	이벤트 원천	조직 Hook, push·pull_request, JSON
Ingress	외부 노출 경계	전용 경로만 Gateway 전달, Activepieces 직접 경로 404
Event Gateway 0.2.0	신뢰·계약 경계	HMAC, allowlist, normalize, 7일 dedup
Activepieces CE 0.86.3	시각적 실행	Catch Webhook → HTTP Action, Run 이력
Chat Adapter	업무 결과 판정	보호 URL, Redis 600 ms slot, success=true
Observer	운영 가시성	최근 15분 Flow 실패 경보

Activepieces Community Edition의 사설 Piece 배포에 의존하지 않도록 Chat Token과 Synology 프로토콜 판정은 서버 내부 Adapter가 소유합니다. Activepieces는 전체 정책·권한·상태를 소유하지 않고 실행 오케스트레이션을 담당합니다.

네트워크 최소 허용

STRICT 네트워크를 유지한 채 automation_egress /28에 제어 API .9와 Chat Adapter .10 두 주소만 SSRF 예외로 허용했습니다.

3. 이벤트 계약과 Flow

입력	조건	HTTP	업무 판정
Ping	Valid HMAC	200	Chat 없음
Push	허용 조직	202	Flow + Chat
PR Merge	closed + merged	202	Flow + Chat
PR Close	merged=false	200	무시
Duplicate	동일 Delivery ID	200	재게시 없음
Bad Signature	HMAC 불일치	401	거부
Direct AP	/api/v1/webhooks/*	404	차단
Chat	2xx + success=true	—	SUCCEEDED

Activepieces Flow

항목	값
Flow	TechFlow - GitHub to Synology Chat v1
상태	ENABLED, LOCKED
Trigger	@activepieces/piece-webhook 0.1.39
Action	@activepieces/piece-http 0.11.14
표현식	trigger.output.body 하위 eventId, data.url, message.text

Flow ID와 Published Version은 저장소의 구조화 계약과 완료 보고서에서 추적합니다. 실제 Secret·세션·인증 응답은 저장소, Issue, PR, PDF에 포함하지 않습니다.

4. 배포 절차와 자산화

단계	수행 내용	보호 장치
1 Build	Gateway 0.2.0 재현 빌드, 25 tests	Image ID 잠금
2 Backup	상태·Source Lock·Runtime Lock 백업	Volume 보존
3 Deploy	Gateway·Ingress 우선, 필요 서비스만 재생성	최소 변경
4 Flow	Catch → Chat Adapter 게시 및 잠금	내부 URL만 사용
5 Canary	Ping·Push·PR, 임시 저장소 Hook	단계별 중단 가능
6 Cutover	임시 Hook 제거 후 조직 Hook 전환	정상 Hook 보존
7 Verify	계약·E2E·Health·Lock·Secret 검사	증거 기록

백업 Label

issue19-predeploy · issue19-ssrf-allowlist · issue19-worker-control-network · issue19-chat-pacing

최종 런타임

항목	검증 결과
서비스	Ingress, Gateway, App, Worker, PostgreSQL, Redis 6/6 healthy
Worker	Polling ready
릴리스	Gateway Image ID를 image-lock.json에 고정
배포 위치	/opt/ablestack-techflow/activepieces
운영 문서	docs/runbooks/github-chat-webhook.md

5. 계약·보안·자동 테스트

ID	검증	결과
T01	유효 GitHub Ping	200
T02	위조 HMAC	401
T03	신규 Push 접수	202
T04	동일 Delivery 재수신	200, 추가 Chat 없음
T05	비병합 PR Close	200, Chat 미호출
T06	외부 Activepieces 직접 Webhook	404
T07	다른 조직 Payload	계약 테스트에서 거부
T08	HTTP 200 + success=false	FAILED 판정
T09	Secret 검사	저장소 배포 자산 49개, 노출 0건
T10	자동 테스트	25건 통과

로그 최소화

원문 GitHub Payload, HMAC 서명, PR 제목, Chat Token을 구조화 로그에 기록하지 않습니다.

재검증 명령은 `deploy/compose/activepieces/scripts/verify-github-chat.py`에 자산화했습니다. 기본 모드는 실제 Chat을 호출하지 않으며, 운영자가 명시적으로 `--live-message`를 지정한 경우에만 게시 검증을 수행합니다.

6. 실제 GitHub E2E 근거

PR	목적	결과
#36	저장소 Hook 기본 Push-PR Merge	PR 성공, Push 411 발견
#37	600 ms 전송 슬롯	이중 Hook 영향 재확인
#38	조직 Hook 단독 최종 검증	PR Merge-Push 모두 성공

초기 실패를 숨기지 않고 Synology 응답 코드 411(create post too fast)을 근거로 Redis 전송 슬롯을 추가했습니다. 최종 전환 전 임시 저장소 Hook을 제거해 동일 Push의 이중 전송 원인도 제거했습니다.

최종 조직 Hook Delivery

이벤트	GitHub HTTP	Activepieces	Chat
PR closed + merged	202	SUCCEEDED	success=true
Push refs/heads/main	202	SUCCEEDED	success=true

다른 조직 Hook 보존

대상	이벤트	Active	최근 결과
kakaowork.com	push	true	HTTP 200
techflow.ablecloud.io	push, pull_request	true	HTTP 202

7. 운영·장애 처리·롤백

순서	조치	확인
1 감지	Activepieces Run, Observer 경보	Event ID로 추적
2 분류	FAILED / UNKNOWN / 계약 거부	원문 Payload 미보존
3 중단	조직 Hook 또는 Flow 비활성화	중복 게시 차단
4 복구	Previous Runtime Lock으로 롤백	Volume:이력 보존

롤백 기준

- 첫 단계는 조직 Hook 비활성화, 두 번째는 github-chat-v1 Flow 비활성화입니다.
- 서비스 롤백은 runtime-lock.previous.json과 rollback-release.sh를 사용합니다.
- 기존 직접 Chat Hook은 업무 실패 상태였으므로 복원보다 비활성 유지가 기본입니다.
- PostgreSQL-Redis-Activepieces Volume과 Flow Run 이력은 삭제하지 않습니다.
- 정상 KakaoWork Hook은 롤백 범위가 아닙니다.

실패 가시성

최근 15분 실패는 flow_failures_warning으로 집계합니다. 담당자·별도 통보 채널이 정해질 때까지 Activepieces Flow Runs와 Observer가 공식 확인 지점입니다.

8. 완료 판정

완료 기준	상태
Push 메시지 정상 게시	완료
PR Merge 메시지 정상 게시	완료
비병합 PR 무시	완료
GitHub HMAC 검증	완료
Delivery ID 중복 억제	완료
HTTP 200와 업무 실패 분리	완료
Activepieces-Observer 실패 가시성	완료
기존 정상 조직 Hook 보존	완료
서버 배포·검증·롤백 절차 자산화	완료
저장소 Secret 노출 0건	완료

Issue #19

구현·배포·조직 Hook 전환·E2E 검증·운영 자산화 범위를 모두 완료했습니다.

추적 자산

docs/plans/issue-19-github-chat-webhook-design.md
docs/decisions/github-chat-webhook-contract.json
docs/runbooks/github-chat-webhook.md
docs/reports/issue-19-github-chat-webhook-validation.md
deploy/compose/activepieces/flows/github-chat-v1.json
deploy/compose/activepieces/scripts/verify-github-chat.py